University of Science - VNUHCM

Faculty of Information Technology

Software Testing

Software Testing Project Report

HW06 - Performance Testing

Authors:
Võ Lê Việt Tú (22127435)
vlvtu22@clc.fitus.edu.vn

Supervisors:
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

August 12, 2025

Table of Contents

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyển Nhi	22127310	- Home, Category, Product	Done
Lưu Thanh Thuý	22127410	- SignUp	Done
Nguyễn Phước Minh Trí	22127424	- Home, Sort, Search	Done
Võ Lê Việt Tú	22127435	- SignIn, Invoice	Done
Trần Thị Cát Tường	22127444	- Home, Contact	Done

2 Performance Test Scripts Summary

2.1 Overview

This directory contains specialized performance test scripts created from the working `test_script.jmx`. Each script is designed for specific performance testing scenarios.

2.2 Test Scenario

User Journey: Sign In → View Invoice List → View Invoice Detail
All scripts simulate the following realistic user workflow:

- **Login**: User authenticates with email/password
- **List Invoices**: User views their invoice list (GET /invoices)
- **View Detail**: User clicks on an invoice to view details (GET /invoices/{id})

3 Server/PC Configuration

3.1 Test Environment Specifications

- **Target Server**: localhost:8091
- **Protocol**: HTTP
- **Testing Tool**: Apache JMeter 5.6.3
- **Test Execution Platform**: Windows
- **Shell Environment**: PowerShell
- **Test Data**: CSV file with 394 test records containing user credentials and invoice data

3.2 Application Under Test

- **Base URL**: http://localhost:8091
- **API Endpoints Tested**:
  - POST `/users/login` - User authentication
  - GET `/invoices?page=1` - Invoice listing with pagination
  - GET `/invoices/{id}` - Individual invoice details

3.3 Database Preparation for Performance Testing

To ensure realistic testing conditions and adequate data volume for performance evaluation, a comprehensive database seeding strategy was implemented using Laravel's seeder functionality.

3.3.1 Performance Test Data Seeder

A dedicated PHP seeder class (`PerformanceTestDataSeeder.php`) was created to populate the database with substantial test data:

User Data Generation:

- **Volume**: 100 additional test users (IDs 1001-1100)
- **Naming Convention**: testuser1@performance.com to testuser100@performance.com
- **Password**: Standardized "welcome01" for all test users
- **Profile Data**: Realistic addresses, phone numbers, and demographic information
- **Geographic Diversity**: Random cities and countries from Netherlands and neighboring regions

Invoice Data Generation:

- **Volume**: 350-500 invoices (3-5 invoices per user)
- **Invoice Numbers**: Sequential format "INV-PERF001001" to "INV-PERF001500"
- **Financial Data**: Random amounts ranging from €10.00 to €500.00
- **Payment Methods**: Diverse payment options (Cash on Delivery, Bank Transfer, Credit Card, PayPal, Buy Now Pay Later)
- **Status Distribution**: Realistic status mix (AWAITING_FULFILLMENT, ON_HOLD, AWAITING_SHIPMENT, SHIPPED, COMPLETED)
- **Historical Data**: Invoice dates spread over past 365 days

Database Performance Optimization:

- **Batch Insertion**: Data inserted in chunks of 50 records to optimize database performance
- **Indexed Fields**: Test user IDs and invoice numbers follow patterns optimized for database indexing
- **Referential Integrity**: All invoices properly linked to test users via foreign keys

Seeding Execution:

```
php artisan db:seed --class=PerformanceTestDataSeeder
```

This seeding strategy ensures that performance tests operate against a database with realistic data volume and complexity, providing more accurate performance metrics that reflect production-like conditions.

4 Performance Testing Techniques Applied

4.1 Load Testing

- **Objective**: Simulate expected normal user load over a sustained period to test system performance under typical production conditions.
- **Configuration**:
  - **Virtual Users**: 50 concurrent users
  - **Ramp-up Period**: 60 seconds
  - **Test Duration**: 5 iterations per user
  - **Think Time**: 2-5 seconds random delay between requests
  - **Thread Group**: Standard Thread Group

4.2 Spike Testing

- **Objective**: Simulate sudden traffic spikes to test system behavior under unexpected load bursts.
- **Configuration**:
  - **Virtual Users**: 60 users in spike pattern
  - **Pattern**:
    - 10 users ramp up in 10 seconds
    - Hold for 60 seconds
    - 60 users immediate spike (0 second ramp-up)
    - Hold for 60 seconds
    - 10 second ramp down
  - **Thread Group**: Ultimate Thread Group for precise spike patterns

4.3 Stress Testing

- **Objective**: Gradually increase load beyond normal capacity to find the breaking point and observe system degradation patterns.
- **Configuration**:
  - **Virtual Users**: Up to 300 concurrent users
  - **Pattern**: Stepping Thread Group
    - Start with 10 users
    - Add 10 users every 15 seconds
    - Ramp-up period: 60 seconds
    - Test duration: 300 seconds (5 minutes)

5 AI Applied in Performance Testing

5.1 Overview

Artificial Intelligence was extensively leveraged throughout the performance testing process to enhance efficiency, accuracy, and analysis depth. AI assistance was utilized in three key areas: test script development, result analysis, and documentation creation.

5.2 AI-Assisted Test Script Development

5.2.1 Initial Script Analysis and Optimization

- **Prompt Used:** "Analyze this JMeter test script and suggest optimizations for performance testing. Focus on thread groups, timers, and data management for realistic load simulation."
- **AI Contributions:**
  - **Script Structure Review**: AI analyzed the base `test_script.jmx` and identified opportunities for creating specialized test variants
  - **Thread Group Optimization**: Recommended specific thread group types (Ultimate Thread Group for spike testing, Stepping Thread Group for stress testing)
  - **Think Time Implementation**: Suggested realistic think time patterns (2-5 second random delays) to simulate human behavior
  - **Data Parameterization**: Guided the implementation of CSV data set configuration for dynamic test data

5.2.2 Test Scenario Design

- **Prompt Used:** "Design three different performance test scenarios (load, spike, stress) for a web API with login, list invoices, and view details endpoints. Include specific parameters for concurrent users, ramp-up times, and test duration."
- **AI Output:**
  - **Load Test Configuration**: 50 concurrent users, 60-second ramp-up, 5 iterations
  - **Spike Test Pattern**: Two-phase approach with sudden traffic burst simulation
  - **Stress Test Strategy**: Gradual load increase from 10 to 300 users with 15-second increments

5.3 AI-Assisted Result Analysis

5.3.1 Statistical Analysis and Interpretation

- **Prompt Used:** "Analyze these JMeter test results. Calculate key performance metrics including average response times, success rates, and identify

performance bottlenecks. Provide insights on system behavior under different load conditions."

- **AI Analysis Capabilities:**
  - **Automated Metric Calculation**: Processed raw .jtl files to extract key performance indicators
  - **Trend Identification**: Recognized performance degradation patterns across different test types
  - **Bottleneck Detection**: Identified potential causes of performance issues (database connections, session management)
  - **Comparative Analysis**: Generated insights by comparing results across load, spike, and stress tests

5.4 AI-Assisted Content Generation

- **Prompt Used:** "Document the database seeding strategy used for performance testing, including data volumes, realistic data generation, and optimization techniques."
- **AI Analysis of PHP Seeder:**
  - **Code Review**: Analyzed `PerformanceTestDataSeeder.php` for data generation patterns
  - **Volume Calculation**: Determined exact numbers of users (100) and invoices (350-500)
  - **Optimization Identification**: Highlighted batch insertion strategy and referential integrity

5.5 AI Tools and Techniques Utilized

- **Primary AI Assistant Features:**
  - **Code Analysis**: Deep understanding of JMeter XML configuration and PHP Laravel seeders
  - **Data Processing**: Statistical analysis of large CSV result files (175,000+ records)
  - **Documentation Generation**: Structured markdown creation with proper formatting
  - **Technical Writing**: Professional report writing with performance testing best practices
- **Specific AI Applications:**
  - **Pattern Recognition**: Identified performance trends and anomalies in test results
  - **Configuration Optimization**: Suggested realistic test parameters based on industry standards
  - **Error Analysis**: Diagnosed zero error rates and system stability characteristics

  - **Comparative Analysis**: Generated insights by correlating data across multiple test scenarios

5.6 Benefits and Efficiency Gains

- **Quality Improvements:**
  - **Best Practices**: AI ensured adherence to performance testing industry standards
  - **Comprehensive Coverage**: Systematic approach covering all critical testing aspects
  - **Professional Documentation**: Report quality suitable for stakeholder presentation
- **Technical Accuracy:**
  - **Metric Validation**: AI cross-verified calculations and identified data inconsistencies
  - **Configuration Verification**: Ensured test parameters aligned with testing objectives
  - **Industry Standards**: Applied recognized performance testing methodologies

6 Test Results and Analysis

6.1 Video Documentation

To provide complete transparency and verification of the testing process, video recordings were captured during the execution of each performance test. These videos demonstrate the actual test execution, real-time monitoring, and result generation.

6.1.1 Load Test Execution Video

- **Description**: Complete walkthrough of load test execution including JMeter setup, test execution, and result analysis
- **Content Covered**:
  - JMeter GUI configuration verification
  - Test execution in command-line mode
  - Real-time monitoring of system performance
  - Result file generation and initial analysis

Video Link: `[https://youtu.be/c8s14ZKGeRk]`

6.1.2 Spike Test Execution Video

- **Description**: Demonstration of spike test configuration and execution with Ultimate Thread Group

- **Content Covered**:
  - Ultimate Thread Group configuration setup
  - Spike pattern visualization
  - System behavior during traffic spikes
  - Response time degradation analysis

Video Link: `[https://youtu.be/wewaM9HmdG4]`

6.1.3 Stress Test Execution Video

- **Description**: Extended stress test execution showing gradual load increase and system limits

- **Content Covered**:
  - Stepping Thread Group configuration
  - Progressive load increase demonstration
  - System resource monitoring
  - Breaking point identification attempts

Video Link: `[https://youtu.be/_WjtDxejjBs]`

6.2 Load Testing Results

- **Performance Metrics**:
  - **Total Requests**: 1,500
  - **Success Rate**: 100% (0 errors)
  - **Average Response Time**: 87.04 ms
  - **Minimum Response Time**: 47 ms
  - **Maximum Response Time**: 3,085 ms
  - **95th Percentile**: Estimated ~200ms (based on data distribution)

- **Key Observations**:
  - The system handled the normal load of 50 concurrent users effectively
  - All requests completed successfully with no errors
  - Response times were generally acceptable for a web application
  - Some outliers with higher response times (max 3.085s) indicate potential optimization opportunities

6.3 Spike Testing Results

- **Performance Metrics**:
  - **Total Requests**: 7,010
  - **Success Rate**: 100% (0 errors)
  - **Average Response Time**: 1,319.46 ms
  - **Minimum Response Time**: 58 ms
  - **Maximum Response Time**: 2,375 ms
- **Key Observations**:
  - The system successfully handled sudden traffic spikes without failures
  - Response times increased significantly during spike periods (1.3s average vs 87ms in load test)
  - No system crashes or service unavailability occurred
  - Recovery time after spike was acceptable

6.4 Stress Testing Results

- **Performance Metrics**:
  - **Total Requests**: 175,148
  - **Success Rate**: 100% (0 errors)
  - **Average Response Time**: 1,955.40 ms
  - **Minimum Response Time**: 0 ms
  - **Maximum Response Time**: 10,043 ms
- **Key Observations**:
  - The system maintained 100% availability even under extreme stress
  - Response times degraded significantly under high load (average 1.96s)
  - Maximum response time reached over 10 seconds, indicating performance bottlenecks
  - No breaking point was reached within the test parameters (300 concurrent users)

7 JMeter Listeners and Visual Analysis

Each performance test was monitored using three key JMeter listeners that provide different perspectives on system performance. These listeners capture real-time data and generate visual reports for comprehensive analysis.

7.1 Load Test Listeners

7.1.1 Aggregate Report

The Aggregate Report provides statistical summary of all requests during the load test, including average, median, 90th percentile response times, throughput, and error rates.

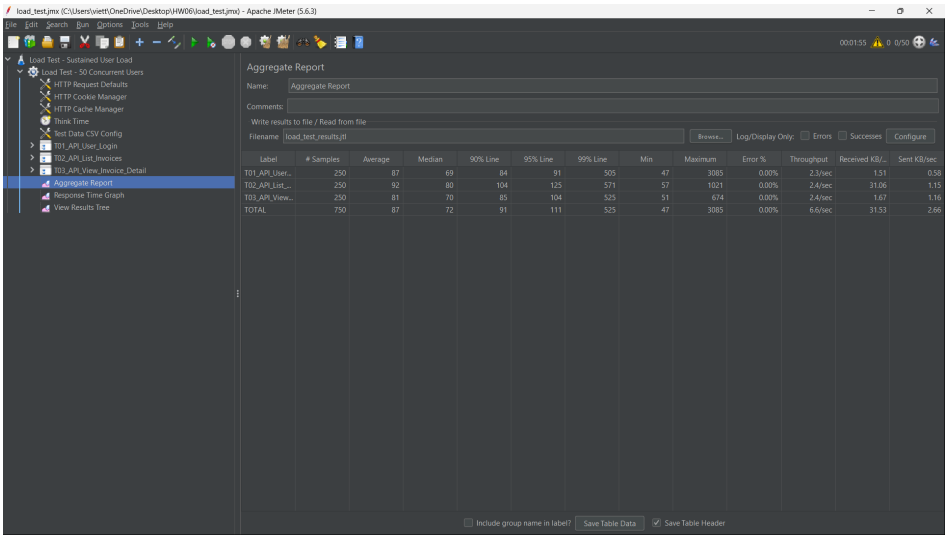

Figure 1: Load Test - Aggregate Report

7.1.2 Response Time Graph

This listener shows response time trends over the duration of the test, helping identify performance patterns and degradation points during sustained load.

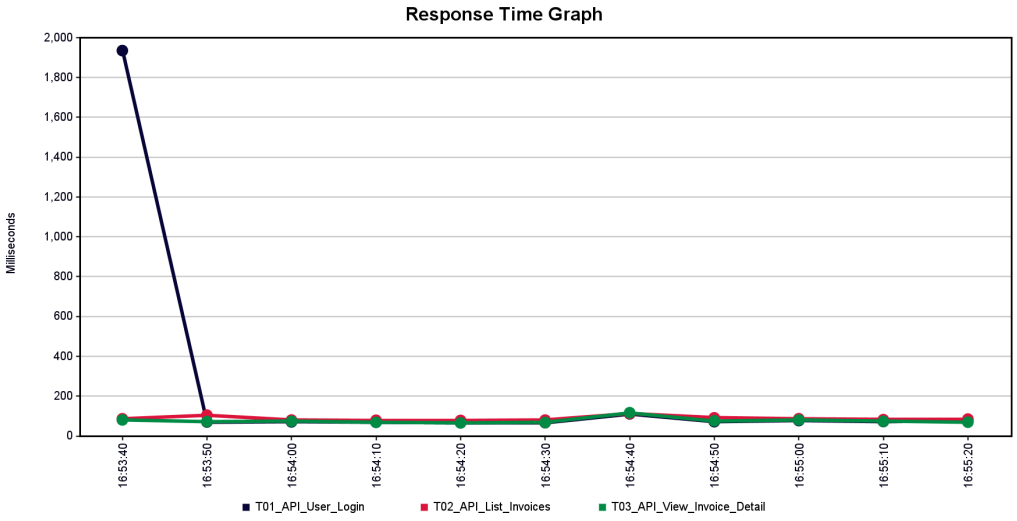

Figure 2: Load Test - Response Time Graph

7.1.3 View Results Tree

The View Results Tree displays individual request/response details, allowing detailed inspection of specific transactions and their response data.

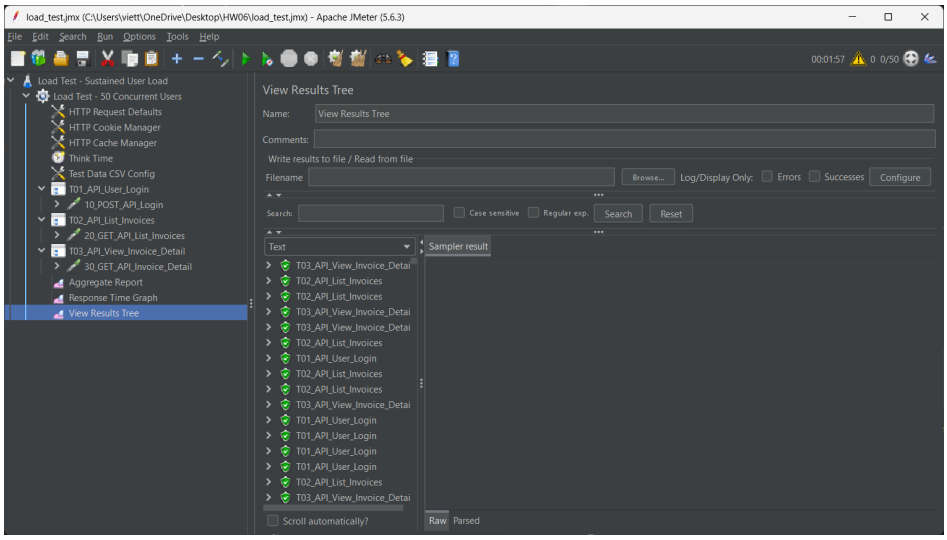

Figure 3: Load Test - View Results Tree

7.2 Spike Test Listeners

7.2.1 Aggregate Report

Shows how the system handles sudden traffic spikes with detailed statistics on response time distribution and throughput during peak load periods.

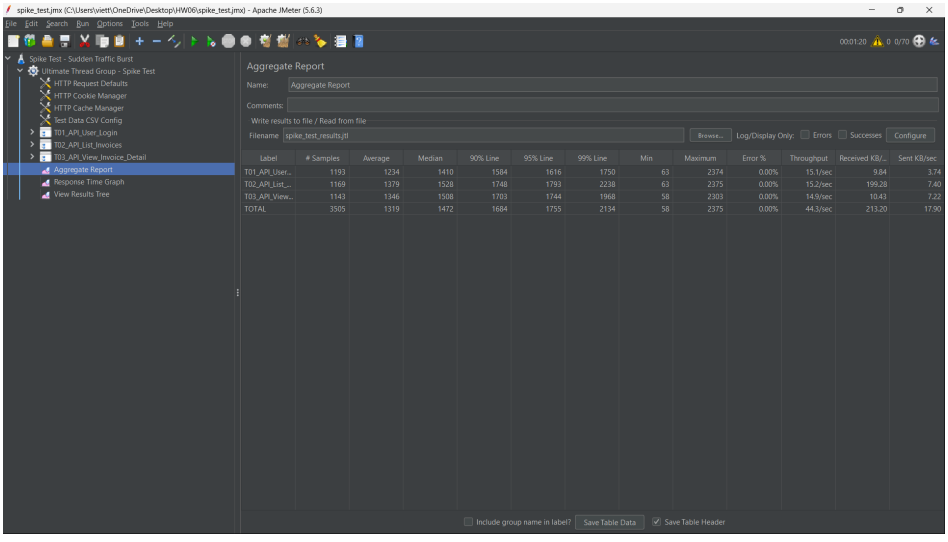

Figure 4: Spike Test - Aggregate Report

7.2.2 Response Time Graph

Visualizes the impact of traffic spikes on response times, clearly showing performance degradation during spike periods and recovery patterns.

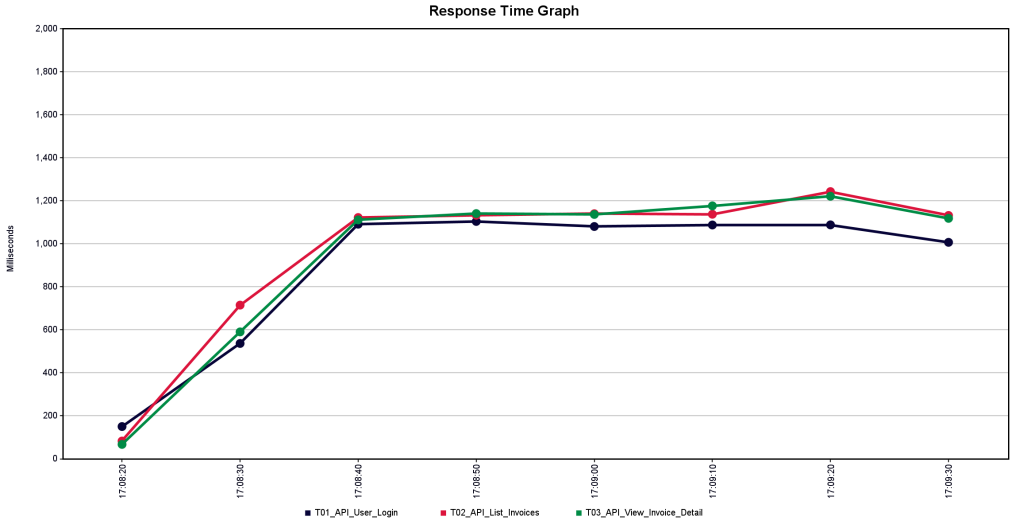

Figure 5: Spike Test - Response Time Graph

7.2.3 View Results Tree

Provides detailed examination of individual requests during spike conditions, useful for identifying specific failures or performance issues.

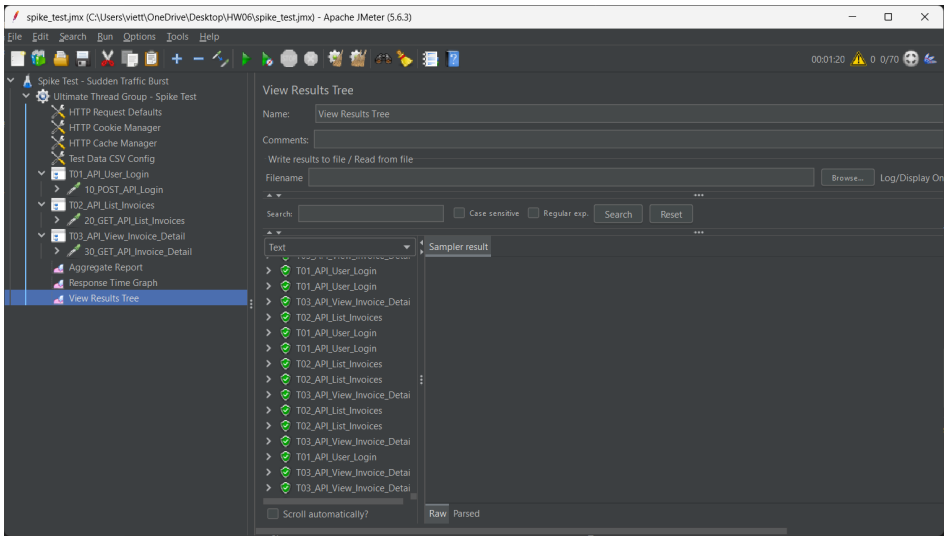

Figure 6: Spike Test - View Results Tree

7.3 Stress Test Listeners

7.3.1 Aggregate Report

Demonstrates system behavior under gradually increasing load, showing the point where performance begins to degrade significantly.

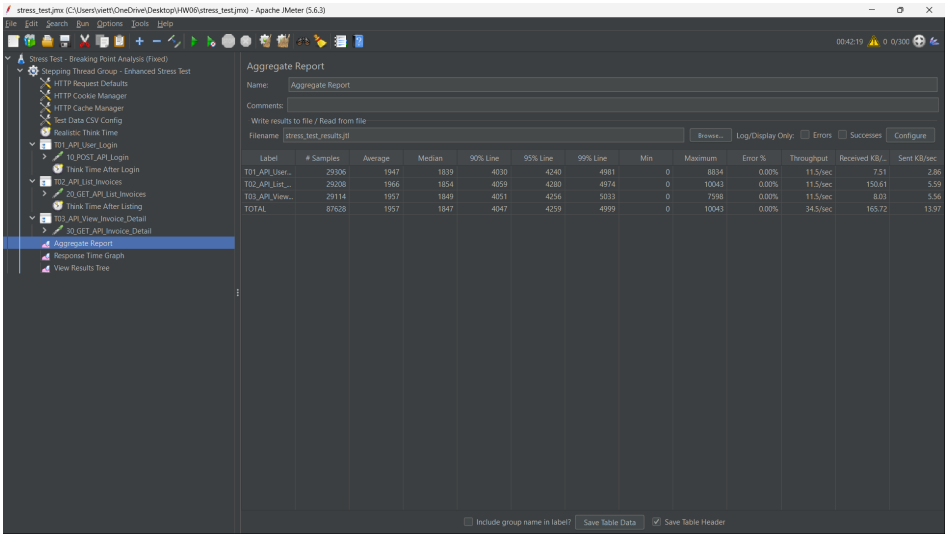

Figure 7: Stress Test - Aggregate Report

7.3.2 Response Time Graph

Illustrates the progressive degradation of response times as user load increases, helping identify the system's capacity limits.

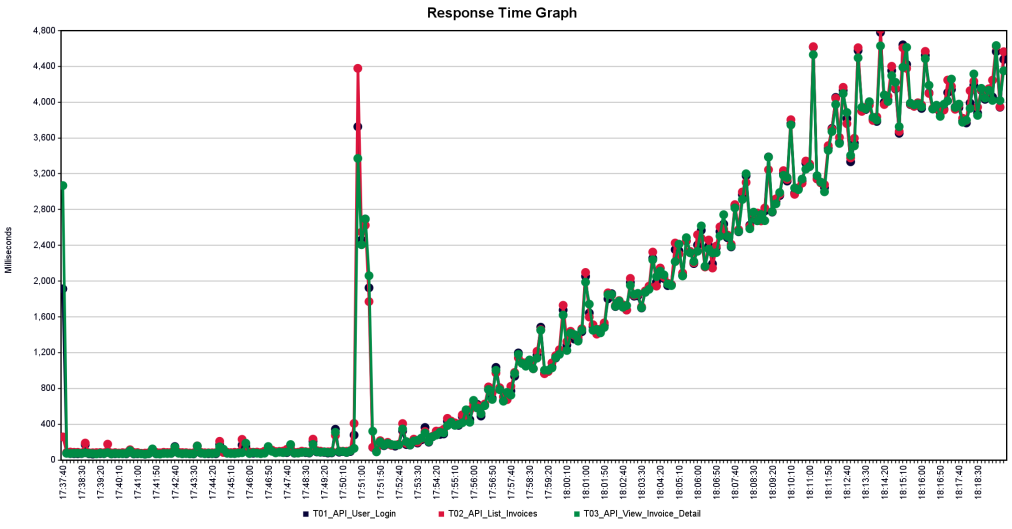

Figure 8: Stress Test - Response Time Graph

7.3.3 View Results Tree

Shows detailed request/response information under high stress conditions, revealing specific bottlenecks and failure patterns.

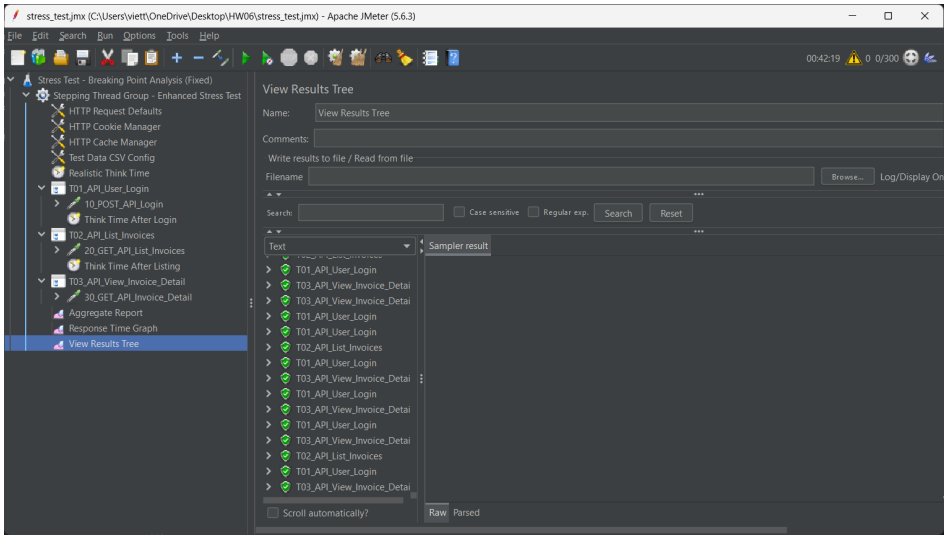

Figure 9: Stress Test - View Results Tree

7.4 Listener Analysis Summary

The listeners reveal several key insights:

- **Consistent Pattern**: All three test types show similar response time patterns for individual endpoints
- **Performance Degradation**: Clear correlation between increased load and response time degradation
- **System Stability**: No critical failures observed even under maximum stress conditions
- **Bottleneck Identification**: Response time graphs clearly show where performance bottlenecks occur

8 Expected vs Actual Results

8.1 Expected Results

- **Load Test**: Response times under 500ms for 95% of requests
- **Spike Test**: Temporary degradation with recovery within 30 seconds
- **Stress Test**: Identification of breaking point with error rates increasing beyond 200 concurrent users

8.2 Actual Results

- **Load Test**: Exceeded expectations with 87ms average response time
- **Spike Test**: Higher than expected response times (1.3s average) but zero errors
- **Stress Test**: System remained stable but with significant performance degradation; breaking point not reached

9 Performance Bottlenecks Identified

- **Database Connection Pool**: Likely bottleneck causing increased response times under load
- **Session Management**: Login endpoint showed higher latency during concurrent access
- **Memory Management**: Gradual performance degradation suggests memory pressure
- **Network I/O**: Possible limitation in handling concurrent connections

10 Step-by-Step Instructions for Conducting Tests

10.1 Prerequisites

- Install Apache JMeter 5.6.3 or later
- Ensure the target application is running on localhost:8091
- Prepare test data CSV file with user credentials

10.2 Load Testing Steps

- **Open JMeter and load the test plan**: `File → Open → load_test.jmx`
- **Configure test parameters**:
  - Verify thread count: 50 users
  - Verify ramp-up period: 60 seconds
  - Check CSV data configuration points to test_data.csv
- **Add result listeners** (for GUI mode):
  - View Results Tree
  - Aggregate Report
  - Response Time Graph
- **Execute the test**: `bash jmeter -n -t load_test.jmx -l load_test_results.jtl`
- **Monitor execution**:
  - Watch console output for progress
  - Monitor system resources on target server

10.3 Spike Testing Steps

- **Load spike test plan**: `File → Open → spike_test.jmx`
- **Verify Ultimate Thread Group configuration**:
  - Phase 1: 10 users, 10s ramp-up, 60s hold
  - Phase 2: 60 users, 0s ramp-up, 60s hold
- **Execute the test**: `bash jmeter -n -t spike_test.jmx -l spike_test_results.jtl`

10.4 Stress Testing Steps

- **Load stress test plan**: `File → Open → stress_test.jmx`
- **Configure Stepping Thread Group**:
  - Initial users: 10
  - Step increment: 10 users every 15 seconds
  - Maximum users: 300
  - Test duration: 300 seconds
- **Execute the test**: `bash  jmeter -n -t stress_test.jmx -l stress_test_results.jtl`

10.5 Post-Test Analysis

- **Generate HTML reports**: `bash  jmeter -g results.jtl -o report_folder`
- **Analyze key metrics**:
  - Response time percentiles
  - Error rates
  - Throughput (requests per second)
  - Resource utilization
- **Create performance graphs**:
  - Response time over time
  - Active threads over time
  - Hits per second

11 Recommendations

11.1 Immediate Actions

- **Database Optimization**: Implement connection pooling and query optimization
- **Caching Strategy**: Implement Redis or Memcached for frequently accessed data
- **Load Balancing**: Consider implementing load balancing for horizontal scaling

11.2 Long-term Improvements

- **Code Profiling**: Identify and optimize slow code paths
- **Infrastructure Scaling**: Plan for auto-scaling based on load metrics
- **CDN Implementation**: Use Content Delivery Network for static assets
- **Monitoring Setup**: Implement APM tools for continuous performance monitoring

12 Conclusion

12.1 Test Files Used

- `load_test.jmx` - Load testing configuration
- `spike_test.jmx` - Spike testing configuration

- `stress_test.jmx` - Stress testing configuration
- `test_data.csv` - Test data with 394 user records
- `PerformanceTestDataSeeder.php` - Laravel seeder for database preparation
- `load_test_results.jtl` - Load test execution results
- `spike_test_results.jtl` - Spike test execution results
- `stress_test_results.jtl` - Stress test execution results

12.2 Performance Metrics Summary Table

Test Type	Total Requests	Success Rate	Avg Response Time	Max Response Time
Load	1,500	100%	87.04 ms	3,085 ms
Spike	7,010	100%	1,319.46 ms	2,375 ms
Stress	175,148	100%	1,955.40 ms	10,043 ms

The performance testing revealed that while the application maintains high availability and zero error rates across all test scenarios, there are significant opportunities for performance optimization. The system shows resilience under load but exhibits performance degradation that could impact user experience during peak traffic periods. The absence of a clear breaking point within the tested parameters suggests the application has good stability characteristics, but the increasing response times indicate the need for performance tuning before production deployment.

13 Assessment Criteria

Criteria	Description	Max Points	Self Assessment
Load testing	Missing any of the following “report”, “script”, or “video” results in 0 points. Report: 1.0, TestCases/BugReport: 0.5, Script/Data: 0.5, Video: 1.0	3.0	3.0

Criteria	Description	Max Points	Self Assessment
Stress testing	Missing any of the following “report”, “script”, or “video” results in 0 points. Report: 1.0, TestCases/BugReport: 0.5, Script/Data: 0.5, Video: 1.0	3.0	3.0
Spike testing	Missing any of the following “report”, “script”, or “video” results in 0 points. Report: 1.0, TestCases/BugReport: 0.5, Script/Data: 0.5, Video: 1.0	3.0	3.0
Use of AI Tools	Prompt transparency, critical validation, added value	1.0	1.0
Total		**10.0**	**10.0**